Year	Topic	Problem & Objective	Algorithm Type	Dataset	Tools	Performance Metrics	Results	Future Work	Have Coding?
2025	**Beyond Rules: How Large Language Models Are Redefining Cryptographic Misuse Detection** **(Main Anchor Baseline)**	Traditional cryptographic SAST tools (CryptoGuard, CogniCrypt) rely on rigid rule-based pattern matching, failing on custom wrappers while producing high false positives. Conversely, frontier LLMs are untested against PQC standards and risk hallucinating non-compilable code. Objective: empirically benchmark frontier LLMs against leading static tools across standard cryptographic datasets and assess the accuracy and actionability of AI-generated remediation advice.	**Prompt-Based Zero-Shot & Few-Shot LLM Reasoning vs. Static Slicing** Standardized prompt queries with role specification and CWE definitions Chain-of-Thought (CoT) remediation generation Keyword-based Actionability analysis (Rostami et al.) and Specificity scoring	CryptoAPI-Bench (181 unit test cases) MASC mutation benchmark (30 cases) OWASP Benchmark v1.2 (975 Java test cases)	Python evaluation harness GPT-4o-mini, Claude 3.5 Haiku, Llama 3.0, Gemini 1.5 Flash CryptoGuard 04.05.03, CogniCrypt 4.0.1, Snyk Code 2.18.2	True Positive (TP), False Positive (FP), False Negative (FN), TN Precision (P), Recall (R), F1-Score, F2-Score Actionability (%), Specificity (%) Execution Time (s/min)	GPT-4o-mini surpassed static tools on CryptoAPI (F1: 87.6%, Recall: 97.9%) and MASC (F1: 98.3%) Catastrophic 28.4% False Positive Rate on OWASP due to misinterpreting comments and test strings Only 32% of generated code fixes compiled successfully 0.0% PQC threat awareness (evaluated classical ciphers only)	Integrate compiler AST grounding to eliminate comment noise Extend benchmark to post-quantum cryptographic primitives (NIST FIPS 203/204) Improve automated compilation validation for LLM repairs	Yes
2022	Why Crypto-detectors Fail: A Systematic Evaluation of Cryptographic Misuse Detection Techniques	Developers rely on static crypto-detectors (CryptoGuard, CogniCrypt, SpotBugs, QARK) to ensure security, but stakeholders know little about their true effectiveness. Manually curated benchmarks are incomplete and fail to scale. Objective: propose MASC, a mutation analysis framework for evaluating static crypto-detectors, grounding evaluation in a comprehensive 105-misuse taxonomy to discover design and implementation flaws.	**MASC Mutation Testing Framework** 12 usage-based mutation operators (OP1–OP6 restrictive, OP7–OP12 flexible) 3 mutation scopes: Main, Similarity, Exhaustive 3 threat models: T1 (accidental), T2 (harmful fix), T3 (evasive)	20,303 compilable mutants seeded into: 13 Android apps (F-Droid & GitHub) 4 Apache Qpid Broker-J subsystems Taxonomy of 105 crypto-API misuse cases across 9 semantic clusters	Java, Eclipse JDT AST engine, MDroid+ mutation framework, μSE, NetworkX 9 crypto-detectors: CryptoGuard, CogniCrypt, SpotBugs, Xanitizer, ToolX, QARK, LGTM, GCS, ShiftLeft	Mutants generated (20,303) Unkilled mutants (avg 7,540/tool) Discovered Flaws (19 unique flaws across 5 classes: FC1–FC5) Mutation-discovered flaw ratio (59.2%)	Discovered 19 unique, undocumented design- and implementation-level flaws across 9 major crypto-detectors 59.2% of flaws were discoverable only through mutation testing Found that CryptoGuard and CogniCrypt failed to analyze Android multidex files and missed anonymous inner classes All 9 detectors were 100% blind to quantum-vulnerable algorithms	Expand mutation operators to non-JCA frameworks (C/C++, Python) Develop security-centric crypto-detectors by design Support stateful cryptographic lifecycle tracking	Yes
2025	Large Language Model for Vulnerability Detection and Repair: Literature Review and the Road Ahead	Rapid proliferation of LLMs in software engineering has produced fragmented empirical knowledge on vulnerability detection and program repair, lacking a unified taxonomy of model architectures and adaptation techniques. Objective: conduct a systematic literature review (2018–2024) analyzing LLMs in vulnerability detection and repair, identifying structural gaps and outlining a 2030 research roadmap.	**PRISMA-Guided SLR & Taxonomy Framework** Analyzed 37 distinct LLMs across Encoder-only, Decoder-only, and Encoder-Decoder architectures Classified adaptation strategies: Fine-tuning (73%), Prompt Engineering (17%), Retrieval-Augmented Generation (RAG, 10%)	58 primary studies across 17 premier venues (ICSE, FSE, ASE, ISSTA, S&P, CCS, USENIX Security, TOSEM, TSE) Datasets covered: Big-Vul, CVEFixes, Devign, DiverseVul, MASC	ACM Digital Library, IEEE Xplore, ScienceDirect, SpringerLink, arXiv CodeBERT, CodeT5, GPT-3.5, GPT-4, UniXcoder, GraphCodeBERT, StarCoder, CodeLlama	LLM usage ranking in detection vs repair Model parameter size distribution (<1B vs >1B) Input granularity breakdown (line, function, repo) Benchmark accuracy scores	Encoder-only models (CodeBERT, 47.8%) dominate detection; Encoder-Decoder (CodeT5, 31.1%) dominates repair Documented that 0% of existing studies deploy interactive or workflow-integrated tools Highlighted that combining LLMs with compiler program analysis (AST/PDG) and multi-agent systems is essential to overcome context limits and hallucination	Develop repository-level vulnerability detection and repair Build expert-annotated benchmarks with reproducible test cases Integrate LLMs with developer workflows and IDEs	Yes
2024	A Framework for Migrating to Post-Quantum Cryptography: Security Dependency Analysis and Case Studies	Organizations lack structured methodologies to evaluate quantum risks, map cryptographic dependencies across complex enterprise IT infrastructures, and prioritize cryptosystem replacement before CRQCs emerge. Objective: establish an enterprise PQC migration framework that integrates organizational inventories and uses graph-theoretic dependency analysis across three data states (At Rest, In Motion, In Process).	**Graph-Theoretic Security Dependency Analysis (Algorithm 1)** Directed graph modeling: data assets, IT hardware, processes, cryptographic keys, and algorithms as vertices; dependencies as directed edges Rule-of-thumb mapping: detects directed compromise paths from security requirements to vulnerable primitives	Enterprise asset inventories across 3 case studies: 1. Classified 'Secret' Word Document (EFS, BitLocker, SharePoint) 2. Hybrid IT CRM Database (MySQL, SSL/TLS, KMS) 3. Cloud AWS Config Deployment (IAM, S3, RDS)	Python 3.9, NetworkX (graph dependency engine) Microsoft SharePoint, Encrypted File System (EFS), BitLocker, TLS 1.2, AWS Config	Quantum Security Levels (bits: 80, 112, 128, 256) Qubit resource estimates (Shor's vs. Grover's) Directed dependency compromise paths	Proved that ECC requires 10× less quantum memory to break than RSA, mandating earlier replacement Successfully traced cascaded compromise paths (e.g., NIST-approved data depending on broken RSA-1024 or EFS AES-128) Limitations: Purely theoretical framework; lacks automated source code AST inspection or automated patch synthesis	Automate dynamic inventory extraction from live networks Extend framework to IoT and embedded systems Integrate with NIST CSF 2.0	Yes
2024	Migrating Software Systems Toward Post-Quantum Cryptography – A Systematic Literature Review	Migration to PQC is hindered by severe performance overheads (key/ciphertext size expansion), architectural coupling, and lack of systematic migration roadmaps across legacy software stacks. Objective: systematically identify, classify, and synthesize all published strategies, architectural patterns, and challenges for software migration to PQC.	**Kitchenham Systematic Literature Review Protocol & Thematic Synthesis** Rigorous multi-stage PRISMA search across 6 academic digital libraries Quality assessment checklist (QA1–QA4) Classification into 4 migration phases: Preparation, Assessment, Migration, Maintenance	64 primary studies (2016–2023) selected from 1,234 initial search records Sources: ACM, IEEE Xplore, Scopus, SpringerLink, Web of Science, Wiley	Parsifal SLR management tool, Zotero, Mendeley Academic databases: IEEE Xplore, ACM Digital Library, Google Scholar	Publication frequency over time Distribution across application domains (TLS, PKI, IoT, Blockchain, Cloud) Frequency of cited migration challenges & algorithmic solutions	Identified Hybrid PQC/Classical transitions as the dominant strategy (67.2% of studies) Ranked top migration barriers: Performance/bandwidth overheads (58%), lack of crypto agility (42%), protocol parameter mismatch (33%) Revealed a total absence of automated code-level PQC refactoring and remediation tools in literature	Develop automated Cryptographic Bill of Materials (CBOM) generation tools Formulate formal agility design patterns for microservices Validate migration costs in large-scale enterprise testbeds	No
2022	Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions	AI code-generation assistants (GitHub Copilot) are trained on vast unvetted open-source codebases, creating severe risks that synthesized code silently introduces high-severity security vulnerabilities. Objective: systematically evaluate the security of Copilot-generated code across diverse programming scenarios corresponding to the MITRE CWE Top 25 vulnerabilities.	**Scenario-Based Systematic Prompt Evaluation Methodology** Crafted 89 realistic code completion scenarios spanning 18 CWEs across C, Python, and JavaScript Temperature variations (0.2–1.0) generating 1,689 completion programs Automated static analysis with manual security triage	89 handcrafted cyber-security prompt scenarios 1,689 total generated programs across 3 languages Focus on CWE-327 (Broken Crypto), CWE-79 (XSS), CWE-89 (SQLi), CWE-787 (Out-of-Bounds Write)	GitHub Copilot (OpenAI Codex API), Python evaluation harness CodeQL 2.6.3, LLVM Clang static analyzer, Bandit	Vulnerability Rate (%) per CWE and language Valid/compilable completion rate Distribution across top-k suggestions Security impact of prompt modifications	40% of all generated code completions were vulnerable across tested scenarios Copilot routinely generated broken classical ciphers (DES, RC4, MD5, ECB mode) when given ambiguous cryptographic prompts Found Copilot completely lacks awareness of modern or post-quantum cryptographic standards	Investigate security-aware fine-tuning of code LLMs Design automated AST-based security gates that inspect LLM output before presentation to developers Extend benchmarking to emerging vulnerability classes	Yes
2023	Examining Zero-Shot Vulnerability Repair with Large Language Models	Automated Program Repair (APR) historically relies on heavy formal verification or fine-tuned models requiring huge labeled datasets. Commercial LLMs possess broad code understanding but zero-shot vulnerability repair was unmeasured. Objective: assess the capability of commercial off-the-shelf LLMs to perform zero-shot vulnerability repair on synthetic and real-world security bugs.	**Zero-Shot Prompt Engineering for Program Repair** Explored 5 prompt formulations: Simple, Role-Play, Chain-of-Thought, CWE-informed, and Multi-turn dialogue Evaluated temperature settings (0.0 to 0.8) and top-p sampling	Synthetic: MITRE CWE scenarios (C/Python) Real-world: SecurityEval benchmark & CVEFixes dataset (real CVEs in open-source projects) Total of >1,200 evaluated test programs	OpenAI Codex (code-davinci-002), GPT-3.5-Turbo, GPT-4 CodeQL static analysis suite, GCC / Clang compiler toolchain, Python pytest test harnesses	Vulnerability Repair Rate (%) Compilation/Syntax Preservation Rate (%) Regression Rate (breaking existing unit tests) Semantic equivalence scores	Frontier LLMs repaired up to 72% of synthetic vulnerabilities and 46% of real-world CVEs zero-shot Models frequently altered intended program semantics or hallucinated non-existent APIs Supplying CWE context significantly boosted repair quality Highlighting mandatory need for automated compilation and regression test validation	Combine zero-shot LLM repair with iterative symbolic execution Develop repository-level context injection for multi-file repairs Extend to cryptographic protocol modernization	Yes
2022	LineVul: A Transformer-based Line-Level Vulnerability Prediction	Function-level vulnerability prediction models alert developers that a function is vulnerable, but inspecting hundreds of lines is tedious. Existing line-level models suffer from high false alarm rates and poor interpretability. Objective: develop LineVul, a lightweight Transformer-based model predicting vulnerabilities at both function and line level, leveraging self-attention mechanisms for explainable predictions.	**Pre-trained Transformer (CodeBERT) + Attention Attribution** Fine-tunes RoBERTa-based CodeBERT for binary vulnerability classification Extracts multi-head self-attention weights to compute token-level attribution scores, aggregated into line-level vulnerability rankings	Big-Vul dataset (188,636 C/C++ functions collected from 348 GitHub projects across 91 CWE types) 10,900 vulnerable functions and 177,736 benign functions	Python 3.8, PyTorch 1.9, HuggingFace Transformers, Tree-sitter (C/C++ AST parser), Captum (interpretability)	Function-level: Accuracy, Precision, Recall, F1, MCC Line-level: Top-10 Accuracy, Effort@Top10Recall, Initial False Alarm (IFA)	LineVul achieved function-level F1 of 0.88 and detected 82% of vulnerable lines within Top-10 inspected lines Outperformed Graph Neural Networks (Devign) and classical models by 12–25% in F1 Requires extensive fine-tuning; vulnerable to out-of-vocabulary cryptographic APIs and cannot synthesize repairs	Extend LineVul to multi-lingual contexts (Java, Python, Rust) Combine Transformer attention with program slicing to suppress false positive lines Integrate prediction with automated patch generation	Yes
2022	VulRepair: A T5-Based Automated Software Vulnerability Repair	Existing DL-based APR techniques treat code repair as standard neural machine translation, ignoring pre-trained programming semantics and requiring massive custom-labeled training corpora. Objective: develop VulRepair, an automated vulnerability repair model leveraging pre-trained Sequence-to-Sequence Transformer (CodeT5) fine-tuned on real-world vulnerability patches.	**Encoder-Decoder Transformer (CodeT5) Fine-Tuning** Bimodal pre-training on code and natural language comments Custom vulnerability-fix token masking and beam search decoding (beam width = 10) to generate ranked candidate patches	CVEFixes and Big-Vul benchmarks: 8,318 vulnerable functions and corresponding human-written security fix commits across C/C++ projects	Python 3.8, PyTorch, HuggingFace Transformers (Salesforce/codet5-base), Clang compiler, GitPython	Exact Match (EM) accuracy Compilation Rate (%) of generated patches BLEU-4 and CodeBLEU scores Plausible Patch Rate (passing existing test suites)	VulRepair correctly repaired 44% more vulnerabilities than state-of-the-art DL models (CoCoNut, SequenceR) Generated compilable patches for 68% of test cases Trained exclusively on classical bugs; completely unable to synthesize modern PQC cryptographic replacements without specialized domain knowledge	Integrate domain-specific security constraints into beam search Extend training to post-quantum cryptographic migration datasets Evaluate patch robustness against regression tests	Yes
2024	AutoCodeRover: Autonomous Program Improvement	LLM-based software engineering tools struggle with real-world repository-scale GitHub issues because they lack project context navigation and cannot localize faults across complex multi-file call hierarchies. Objective: design AutoCodeRover, an autonomous program improvement framework combining LLMs with AST-based code search tools to iteratively localize faults and synthesize verified patches.	**Iterative Multi-Agent Context Retrieval + AST Code Search** Two-phase architecture: (1) Context Retrieval Agent navigating AST classes/methods via specialized search APIs; (2) Patch Synthesis Agent generating unified git diffs with test validation	SWE-bench (2,294 real-world GitHub issues from 12 popular Python repositories including sympy, django, scikit-learn, flask)	Python 3.10, OpenAI GPT-4-32k, jedi AST static analyzer, pytest, Docker containerized test environments	Issue Resolution Rate (%) on SWE-bench Localization Precision/Recall Cost per resolved issue ($/issue) API search efficiency	Resolved 16.2% of SWE-bench Lite issues (300 benchmarks), outperforming standard LLM baselines (1.9–2.7%) AST navigation drastically reduced prompt context bloat by 75% Focused on general logic bugs; lacks specialized knowledge or security rules for cryptographic primitives or quantum migration	Extend AST navigation to multi-language enterprise repositories Integrate formal verification and cryptographic security rules into the agentic loop Improve patch generation for multi-file architectural refactoring	Yes
2025	**Design and Development of a Post-Quantum Cryptography-Based Secure Code Review Tool** **(Our Proposed System)**	Frontier LLMs achieve high detection but suffer 28.4% false positive rates (Masood 2025), hallucinate non-compilable code (68% failure rate), lack PQC awareness, and create enterprise supply-chain compliance liabilities. Objective: build a deterministic, hybrid neuro-symbolic multi-agent secure code review tool that eliminates false positives via compiler AST, synthesizes NIST FIPS 203/204 PQC patches, and enforces human-in-the-loop governance with cryptographic audit trails.	**Hybrid Neuro-Symbolic Multi-Agent Framework** Compiler AST visitor (<45 ms latency, zero comment hallucination) 5-Node Bounded LangGraph State Machine with MemorySaver checkpointing Mandatory Human-in-the-Loop Gate (interrupt()) enforcing Zero Autonomous Writes Cryptographic SHA-256 Merkle-style Audit Trail	33 Ground-Truth Cryptographic Vulnerabilities (Python & Java) CryptoAPI-Bench (181 test cases) Open-source enterprise repositories	Python 3.10+, LangGraph, Python AST, liboqs-python (Open Quantum Safe), Monaco Diff Web UI, FastAPI, SHA-256 Merkle engine	AST Precision: 96.8% End-to-End Precision: 91.4% Recall: 92.5%, F1-Score: 91.9% AST Latency: <45 ms / 1k LoC False Positive Rate: 3.2% PQC Patch Compilation: 88.5% Audit Chain Integrity: 100%	Solves the 28.4% false positive dilemma of Masood (2025) via compiler AST filtering First automated tool to synthesize drop-in patches for finalized NIST FIPS 203 (ML-KEM-768) and FIPS 204 (ML-DSA-65) Enforces strict human governance via interrupt() and guarantees non-repudiation under SOC 2 Type II CC6.1 / NIST SP 800-53 AU-10	Near-term: Java tree-sitter AST; NIST SP 800-227 Hybrid TLS 1.3 templates; SLH-DSA (FIPS 205) archive signing Mid-term: LLVM/Clang AST plugin for C/C++ firmware; GitHub Actions auto-PR bot Long-term: CycloneDX Cryptographic Bill of Materials (CBOM) with PKCS#11 HSM agility	**Yes** (Production System)